Caso práctico

ReservaUPLA: Sistema de Gestión de Reservas de Ambientes Universitarios

1. Situación problemática

La Universidad Peruana Los Andes administra diversos ambientes institucionales, tales como laboratorios de cómputo, auditorios, salas de reuniones y laboratorios especializados. Actualmente, las reservas de estos ambientes se realizan mediante mensajes, llamadas o documentos físicos, lo que ocasiona duplicidad de reservas, pérdida de solicitudes, falta de trazabilidad y dificultad para conocer la disponibilidad real de los ambientes.

Frente a esta problemática, se requiere desarrollar una aplicación web denominada **ReservaUPLA**, orientada a registrar, validar, consultar y visualizar reservas de ambientes universitarios.

2. Propósito del caso práctico

El estudiante deberá desarrollar progresivamente un prototipo funcional de software aplicando los contenidos trabajados desde la semana 01 hasta la semana 09 de la asignatura Construcción de Software.

3. Reto del estudiante

Desarrollar una solución web que permita gestionar reservas de ambientes universitarios, aplicando fundamentos de construcción de software, estándares de desarrollo, validación de datos, reutilización de componentes, refinamiento de código y construcción de módulos de salida de datos.

4. Actores sugeridos

El estudiante deberá analizar y considerar como mínimo los siguientes actores:

Actor	Responsabilidad
Solicitante	Registra solicitudes de reserva.
Administrador	Aprueba o rechaza reservas.

Responsable de ambiente	Consulta el uso del ambiente asignado.

5. Requerimientos funcionales mínimos

El estudiante deberá implementar como mínimo los siguientes requerimientos:

Código	Requerimiento
RF-01	Registrar usuarios solicitantes.
RF-02	Registrar ambientes universitarios.
RF-03	Registrar solicitudes de reserva.
RF-04	Validar disponibilidad del ambiente antes de registrar la reserva.
RF-05	Listar reservas registradas.
RF-06	Consultar detalle de una reserva.
RF-07	Cambiar el estado de una reserva: Pendiente, Aprobada, Rechazada o Cancelada.
RF-08	Mostrar indicadores básicos de reservas en un dashboard.

6. Reglas de negocio mínimas

El estudiante deberá implementar las siguientes reglas:

Código	Regla de negocio
RN-01	No se debe permitir más de una reserva aprobada para el mismo ambiente, fecha y horario.
RN-02	Toda reserva debe registrar fecha, hora de inicio, hora de fin, ambiente, solicitante y motivo.
RN-03	La hora de fin debe ser mayor que la hora de inicio.
RN-04	Las reservas solo podrán realizarse entre las 07:00 y 22:00 horas.
RN-05	Solo el administrador puede aprobar o rechazar una reserva.
RN-06	Una reserva rechazada no podrá modificarse.

7. Modelo de datos referencial

El estudiante podrá ampliar el modelo según su análisis, pero deberá considerar como mínimo las siguientes entidades:

Usuario

Campo	Tipo sugerido
Id	UUID
NombreCompleto	VARCHAR(150)
Correo	VARCHAR(150)
Rol	VARCHAR(50)
Activo	BOOLEAN

Ambiente

Campo	Tipo sugerido
Id	UUID
Nombre	VARCHAR(100)
Tipo	VARCHAR(50)
Capacidad	INTEGER
Ubicacion	VARCHAR(100)
Estado	VARCHAR(30)

Reserva

Campo	Tipo sugerido
Id	UUID
Fecha	DATE
HoraInicio	TIME

HoraFin	TIME
Motivo	TEXT
Estado	VARCHAR(30)
UsuarioId	UUID
AmbienteId	UUID

8. Actividades que debe desarrollar el estudiante

Semana	Tema aplicado	Actividad del estudiante	Evidencia esperada
01	Fundamentos de construcción de software	Analizar el problema, actores, necesidades y alcance del sistema.	Documento breve de análisis.
02	Ciclo de vida del software	Seleccionar y justificar el modelo de ciclo de vida a utilizar.	Justificación técnica del modelo elegido.
03	Estándares de construcción de software	Definir estándares de codificación, nombres, carpetas, commits y ramas.	Manual breve de estándares del proyecto.
04	Aplicación de fundamentos y estándares	Crear la estructura inicial del proyecto backend, frontend y base de datos.	Repositorio inicial organizado.
05	Módulos de entrada de datos	Construir formularios para registrar usuarios, ambientes y reservas.	Formularios funcionales.
06	Validación y captura de datos	Implementar validaciones en frontend y backend.	Formularios con validaciones aplicadas.

07	Reutilización de código	Crear componentes reutilizables para formularios y mensajes.	Componentes reutilizables implementados.
08	Refinamiento de código	Refactorizar formularios, servicios, controladores y validaciones.	Código reorganizado y documentado.
09	Módulos de salida de datos	Construir listado de reservas y dashboard básico.	Módulo de consulta y dashboard funcional.

9. Componentes mínimos sugeridos

El estudiante deberá implementar componentes reutilizables como:

- TextInput
- SelectInput
- DateInput
- TimeInput
- Button
- AlertMessage
- ModalConfirmacion
- TableList
- CardIndicador

10. Indicadores mínimos del dashboard

El dashboard deberá mostrar como mínimo:

Indicador	Descripción
Total de reservas	Cantidad total de reservas registradas.
Reservas pendientes	Cantidad de reservas en espera de aprobación.

Reservas aprobadas	Cantidad de reservas aprobadas.
Reservas rechazadas	Cantidad de reservas rechazadas.
Ambiente más utilizado	Ambiente con mayor número de reservas.

11. Producto que debe entregar el estudiante

El estudiante deberá presentar:

1. Repositorio del proyecto.
2. Backend funcional.
3. Frontend funcional.
4. Base de datos implementada.
5. Evidencia de validaciones.
6. Capturas del dashboard.
7. Informe técnico breve.
8. Sustentación del avance desarrollado.

12. Criterios de evaluación

Criterio	Puntaje
Análisis del problema, actores y alcance	2 pts
Justificación del ciclo de vida	2 pts
Aplicación de estándares de construcción	2 pts
Implementación de formularios de entrada	3 pts
Validaciones frontend y backend	3 pts
Reutilización de componentes	2 pts
Refinamiento de código	2 pts
Módulos de salida y dashboard	3 pts

Presentación y sustentación	1 pt
Total	**20 pts**

13. Pregunta de reflexión final

Explique cómo la aplicación de estándares, validación de datos, reutilización de componentes y refinamiento de código contribuye a mejorar la calidad, mantenibilidad y evolución del sistema ReservaUPLA.